CoWERA — Implementation Plan & Status

Performance refactor + committor-based CV extension

Repository: TeamSuman/CoWERA · Branches: **devel** (PR #2) and **committor** · Updated: 2026-06-20

Status legend: **[DONE]** implemented & CPU-tested | **[IN PROGRESS]** partially implemented | **[FUTURE]** planned

---

1. Overview & context

CoWERA (J. Chem. Phys. 164, 134111, 2026) is a binless weighted-ensemble resampling algorithm built on the wepy/OpenMM stack. It ranks walkers by a sign-based temporal-coherence phase. This effort has two streams: (A) fix bugs and optimize the existing method for multi-GPU workstation performance, and (B) extend CoWERA with a learned **committor** q(x) as the resampling criterion — the provably optimal reaction coordinate.

Guiding constraints: keep the published **cowera/** path the default; land changes as phased, reviewed commits; provide CPU-only unit tests and benchmarks (no GPU/OpenMM in the dev environment), deferring GPU/protein-scale validation to the simulation host.

2. Architecture & branches

- **devel** (PR #2): performance refactor of the core cowera/ modules.
- **committor** (off devel): opt-in `Scripts/cowera_committor/` subpackage; the committor is a different progress coordinate feeding the same phase→intensity machinery, so it required no change to the resampler core.
- Shared prerequisite: performance Phase 3 (persistent per-GPU OpenMM contexts + a runner CV/burst hook) unblocks both the multi-GPU speedup and the committor shooting phase.

---

3. Part A — Performance refactor (branch devel, PR #2)

Phase 1 — Hot-path optimizations + bug fixes *[DONE]*

Bug fixes (with regression tests):

- Operator-precedence bug in the changepoint/relevant-history window.
- Divide-by-zero → NaN when all phases or all I0 weights are equal, and zero-interference; now guarded (scale_phases / scale_weights / compute_intensity).
- Empty-segment phase → NaN; broken except-fallbacks in features.py; unified the RNG.

Optimizations:

- Cache native contacts + RMSD references by file (were rebuilt per walker per cycle).
- Removed the per-pair MDAnalysis Universe construction; vectorized Q core (~150x vs loop).
- Parallelized the per-walker phase loop (joblib).

Files: cowera/features.py, cowera/metric.py, cowera/resampler.py; Scripts/tests/ (+bench_hotpaths.py); docs/ANALYSIS_AND_OPTIMIZATION.md, docs/REPRODUCING_THE_PAPER.md.

Phase 2 — In-memory CV history *[DONE]*

- cowera/cv_history.py (CVHistory): per-walker CV series in memory, incremental update, clone/merge reindexing (mirrors update_dcd_files), warp-reset detection, optional window.
- metric.py decoupled analysis from I/O (phase_from_projection / intensity_from_projections); legacy disk methods kept as wrappers; resampler uses the in-memory path by default.

Validation note: the integrated resampler path needs host validation before merging to main.

Phase 3 — Persistent per-GPU OpenMM contexts *[FUTURE]*

- Replace TaskMapper (rebuilds a CUDA context every walker every cycle) with persistent WorkerMapper + per-process Context cache (setState → step → getState).
- Trim getState to positions + box only; add a runner CV/burst hook (enables committor M4).

Phase 4 — Algorithmic accelerations *[FUTURE]*

- GPU-inline CV via OpenMM CustomCVForce; online/incremental changepoint detection;
- lazy O(N^2)→on-demand merge distance; overlap CPU resampling with GPU propagation;
- optional probabilistic merge survivor P(keep i)=w_i/(w_i+w_j) for strict WE exactness.

---

4. Part B — Committor extension (branch committor)

M1 — Committor core *[DONE]*

- featurizer.py (SE(3)-invariant), committor_model.py (dependency-free MLP, manual backprop; variational/semigroup + boundary + AIMMD + interpolant losses, WE-weighted; GVP-GNN backend can drop in behind the same interface), committor_data.py (WE-weighted buffers), toy_systems.py (double well + exact committor).
- Validation: finite-difference gradient check; double-well committor MAE ~ 0.014; histogram test (mean p_B 0.53). Executed tutorial notebook + 6 figures.

M2 — Delta q live resampling path *[DONE]*

- CommittorDistances(Calculate_Distances): q via model inference (stored in CVHistory); continuous Delta q = q(T) - q(T-tau) replaces the sign-based phase; same intensity machinery.
- Committor-augmented merge distance alpha*D_RMSD + (1-alpha)*|q_i-q_j|. No change to the resampler core.

M3 — Cold-start bootstrap + on-the-fly training *[DONE]*

- phase_manager.py (BootstrapPhaseManager): per-phase loss weights, conservative-merge band, retrain timing, segment routing, phase-advancement criteria. bootstrap.py (structural interpolants). committor_train.py (routing/assembly/retrain). committor_resampler.py, config_committor.yml, run_cowera_committor.py.
- Cold-start demo: q sharpens from the structural seed (Phase 0, MAE ~0.06) to the true committor on dynamical data (Phase 1, MAE ~0.02).

Cold-start Phase 2/3 of the protocol *[IN PROGRESS]*

- Buffers, sliding window, histogram-test and advancement logic exist; the shooting (AIMMD) data generation and the mature self-consistent loop (JSD convergence) are not yet wired.

M4 / shooting-point (AIMMD) augmentation *[FUTURE]*

- Harvest reactive fragments at first B-touch; launch short forward/backward bursts near q~0.5; feed the ShootingBuffer. Depends on the performance Phase 3 runner CV/burst hook.

Stage 2 — GVP-GNN backend + live retraining *[FUTURE]*

- SE(3)-equivariant GVP-GNN behind the CommittorModel interface (torch-geometric); async retraining thread; chignolin / Trp-cage benchmarks vs the published numbers.

Stage 3 — Triple output *[FUTURE]*

- Rate + committor surface + free energy along q (WE-weighted); Kramers cross-check; per-atom importance / interpretability.

Stage 4 — Ligand binding + multi-state committor *[FUTURE]*

- Heterogeneous protein-ligand graphs; online state detection (TICA/PCCA+) and vector/local committor (softmax head and L1 merge are already designed-in); pathway-decomposed rates (TPT).

---

5. Validation status

- **[DONE]** 87 CPU unit tests pass (no GPU/MD/torch): algorithm cores, bug-fix edge cases, CVHistory, committor model gradient check + double-well recovery, Delta q path, phase manager, bootstrap, training orchestration; plus micro-benchmarks and an executed tutorial notebook.

- [FUTURE] Deferred to the simulation host: the integrated in-memory resampler path; committor metric/resampler with real trajectories; multi-GPU performance (Phases 3-4); chignolin / Trp-cage kinetics vs Tables II-III; committor-guided vs phi-guided comparison.

6. Dependencies & risks

- Near-term committor work uses only numpy/scipy (torch confined to the future GVP-GNN backend; cowera/ runs without torch).
- Stage 2+ adds torch / torch-geometric; Stage 4 adds rdkit.
- Sequence risk: committor M4 (shooting) and GPU-batched inference need the runner hook from performance Phase 3 — schedule that first.
- Statistical validity: on-the-fly retraining does not bias WE (exactness comes from weight conservation); the criterion affects efficiency only.

---

7. File map

Area	Files	Status
Perf — core	cowera/features.py, metric.py, resampler.py, cv_history.py	DONE
Perf — tests/docs	tests/test_*; bench_hotpaths.py; docs/ANALYSIS*, REPRODUCING*	DONE
Perf — runner	wepy/runners/openmm.py, run_cowera.py (persistent ctx)	FUTURE
Committor — core	cowera_committor/{featurizer,committor_model,committor_data,toy_systems}.py	DONE
Committor — resamp	cowera_committor/{committor_metric,committor_resampler}.py	DONE
Committor — boot	cowera_committor/{phase_manager,bootstrap,committor_train}.py	DONE
Committor — entry	run_cowera_committor.py, config_committor.yml	DONE
Committor — tests	tests/test_committor_*.py; Analysis/CoWERA_Committor_Tutorial.ipynb	DONE
Committor — shoot	cowera_committor/shooting.py (AIMMD)	FUTURE
Committor — GNN	cowera_committor/gvp_committor.py (backend)	FUTURE

---

8. Recommended next step

Do performance **Phase 3** (persistent per-GPU OpenMM contexts + the runner CV/burst hook) on the devel line. It is the highest-leverage multi-GPU change and is the shared prerequisite that unblocks the committor **M4** shooting phase. Then validate the in-memory (Phase 2) and committor (M1-M3) integrated paths on the GPU host against the chignolin / Trp-cage benchmarks.